

Guia deCodigo para Sistemas Empresariales Modulares

Arquitectura, seguridad, despliegue en Hostinger, modulos funcionales y mejoras para
construir aplicaciones operativas para pequenas y medianas empresas.

Generado el 11 de junio de 2026

Guia tecnica reutilizable

Documento para que otra IA, equipo tecnico o desarrollador entienda como construir, desplegar, asegurar y ampliar una aplicacion web de gestion de procesos empresariales usando una arquitectura simple compatible con Hostinger.

Este documento no depende de una industria especifica. Usa terminos generales como usuarios, equipo, clientes, servicios, reservas, tareas, pagos, notificaciones y reportes.

Indice

- 1 Objetivo del sistema
- 2 Arquitectura recomendada
- 3 Modulos funcionales
- 4 Modelo de datos base
- 5 API y flujos principales
- 6 Frontend y experiencia de usuario
- 7 Seguridad minima y seguridad avanzada
- 8 Despliegue en Hostinger
- 9 Pruebas, observabilidad y mantenimiento
- 10 Mejoras futuras para empresas
- 11 Instrucciones para otra IA

1. Objetivo del sistema

El objetivo es construir una aplicacion web que ayude a una empresa pequena o mediana a administrar procesos diarios desde un panel central. El sistema debe permitir operar sin depender de hojas de calculo dispersas, mensajes manuales o registros duplicados.

En terminos generales, el sistema debe cubrir tres necesidades:

- Gestion operativa: crear usuarios, asignar trabajo, administrar agendas, controlar estados y registrar actividad.
- Autoservicio: permitir que clientes, miembros del equipo o proveedores hagan solicitudes, consulten informacion y reciban notificaciones.
- Control de negocio: mostrar metricas, reportes, pagos, historial y datos accionables para tomar decisiones.

2. Arquitectura recomendada

Para una primera version comercial orientada a PYMES, conviene una arquitectura simple: servidor Node.js, frontend HTML/CSS/JavaScript o framework ligero, API interna, sesiones seguras, almacenamiento persistente y despliegue en hosting administrado.

Estructura de carpetas sugerida

```
project/
  server.js          # Servidor Node.js: API + archivos estaticos
  package.json       # Scripts, version de Node y metadatos
  index.html         # Aplicacion principal o entrada SPA
  admin.html         # Redireccion o entrada de panel administrativo
```

```

css/
  base.css          # Tokens visuales, reset, layout base
  app.css           # Componentes generales
  admin.css         # Estilos de panel interno
js/
  app.js            # Renderizado, rutas y eventos del frontend
  auth.js           # Cliente de login/logout y sesion local
  users-data.js     # Cache cliente sincronizada con API
  data.js           # Catalogos publicos: servicios, textos, opciones
assets/
storage/
  cd/
    data/           # JSON persistente en MVP
    backups/        # Backups automaticos
    uploads/        # Archivos privados o semi-privados
docs/              # Guías, manuales y documentacion tecnica

```

Principios de diseno tecnico

- Separar presentacion y negocio: el frontend no decide permisos criticos ni valida contraseñas.
- Servidor como autoridad: el servidor crea sesiones, filtra datos por rol y registra acciones.
- Configuración por entorno: rutas, secretos y claves deben venir de variables de entorno.
- Persistencia predecible: cada colección tiene un archivo o tabla clara, con backups y migraciones.
- Dominio adaptable: los nombres visibles pueden cambiar por industria, pero el modelo interno debe mantenerse consistente.

3. Modulos funcionales

Un sistema empresarial de este tipo se construye como una colección de módulos conectados. Cada módulo debe tener datos, vistas, permisos, acciones y eventos de auditoria.

Modulo	Funcion	Usuarios tipicos
Autenticacion	Login, logout, sesiones, recuperacion de acceso y control por rol.	Admin, equipo, cliente
Panel administrativo	Vista central para gestionar usuarios, servicios, agenda, pagos, reportes y configuracion.	Admin, gerente
Gestion de equipo	Crear miembros, asignar identificadores, credenciales, especialidades, disponibilidad y estado.	Admin, RRHH, operaciones
Gestion de clientes	Registrar clientes, historial, preferencias, datos de contacto y seguimiento.	Admin, ventas, soporte
Servicios o productos	Catalogo configurable con precios, duraciones, categorias, recursos requeridos y disponibilidad.	Admin, cliente
Reservas, tareas o solicitudes	Crear trabajo operativo, asignarlo a un responsable, cambiar estado y registrar notas.	Cliente, equipo, admin
Calendario y turnos	Disponibilidad, turnos, ausencias, bloqueos y conflictos.	Equipo, admin
Pagos	Registro de pagos, estado, conciliacion, recibos e integracion con pasarelas.	Admin, finanzas, cliente
Notificaciones	Mensajes internos, email, SMS, WhatsApp, recordatorios y alertas de cambios.	Todos
Reportes	Metricas de actividad, ingresos, productividad, retencion, tiempos y conversion.	Admin, gerencia

Roles base

Administrador

Controla configuracion, usuarios, permisos, reportes, pagos, agenda global y datos maestros.

Equipo interno

Recibe asignaciones, actualiza estados, ve su propia agenda y responde notificaciones.

Cliente

Crea solicitudes, consulta historial, actualiza datos, paga y recibe confirmaciones.

Operador invitado

Puede crear una solicitud publica sin cuenta, con datos minimos y seguimiento limitado.

4. Modelo de datos base

Para un MVP, los datos pueden vivir en archivos JSON persistentes. Para produccion con multiples clientes, mayor volumen o auditoria estricta, se debe migrar a una base SQL. La forma de los datos debe disenar esa migracion desde el principio.

Colecciones recomendadas

Coleccion	Campos importantes	Notas
admins	id, username, passwordHash, email, status	No exponer hashes al frontend. Mantener cuentas de emergencia configurables.
staff	id, staffCode, username, profile, availability	El identificador operativo ayuda a enrutar tareas o reservas al responsable correcto.
customers	id, name, email, phone, preferences	Debe cumplir politicas de privacidad y minimizacion de datos.
services	id, name, price, duration, category	Debe ser editable desde admin cuando el negocio madure.
bookings o tasks	id, customerId, staffId, staffCode, date, status	Guardar tanto ID como codigo operativo facilita auditoria y reportes.
roster	id, staffId, date, startTime, endTime, status	Usado para disponibilidad, ausencias, turnos y conflictos.
payments	id, bookingId, amount, provider, status	Separar pago de solicitud permite conciliacion y reintentos.
sessions	token, userId, role, expiresAt	Usar cookies HttpOnly. Limpiar sesiones vencidas.
activityLog	actorId, actorRole, action, entityId, createdAt	Necesario para soporte, auditoria y trazabilidad.

Ejemplo generico de una solicitud operativa

```
{
  "id": "task001",
  "customerId": "c001",
  "customerName": "Cliente Demo",
  "staffId": "s001",
  "staffCode": "EMP-001",
  "serviceId": "svc001",
  "serviceName": "Servicio principal",
  "date": "2026-06-20",
  "time": "10:00",
  "duration": 45,
```

```

    "price": 120,
    "status": "pending",
    "paymentStatus": "unpaid",
    "source": "website",
    "notes": "",
    "createdAt": "2026-06-11T00:00:00.000Z",
    "updatedAt": "2026-06-11T00:00:00.000Z"
  }

```

5. API y flujos principales

La API debe ser pequena, consistente y facil de auditar. Todas las escrituras importantes deben pasar por el servidor, no por manipulacion directa de archivos desde el frontend.

Endpoints base

Endpoint	Metodo	Uso	Permiso
/api/health	GET	Verifica que el servidor y storage esten activos.	Publico
/api/bootstrap	GET	Carga datos visibles segun sesion.	Publico o autenticado
/api/auth/login	POST	Valida credenciales, crea cookie HttpOnly y retorna sesion sanitizada.	Publico
/api/auth/logout	POST	Invalida la sesion actual.	Autenticado
/api/users/staff	POST/PATCH/DELETE	Crea, edita o desactiva usuarios del equipo.	Admin
/api/users/customers	POST/PATCH/DELETE	Gestiona clientes.	Admin o propietario
/api/bookings o /api/tasks	GET/POST	Lista o crea solicitudes operativas.	Segun rol
/api/payments	POST/PATCH	Registra pagos y actualiza estado financiero.	Admin o sistema
/api/notifications	POST/PATCH	Envia mensajes y marca notificaciones como leidas.	Segun rol

Flujo de login

- 1 El usuario envia username, password y rol esperado.
- 2 El servidor normaliza el username y busca en la coleccion correcta.
- 3 La contrasena se compara contra passwordHash con PBKDF2, bcrypt o Argon2.
- 4 Si es correcto y el usuario esta activo, se crea una sesion con expiracion.
- 5 El servidor devuelve una cookie HttpOnly, SameSite y Secure en HTTPS.
- 6 El frontend guarda solo una copia sanitizada de la sesion para navegacion, nunca hashes.

Flujo de creacion de solicitud

- 1 El cliente o admin selecciona servicio, fecha, hora y responsable.
- 2 El servidor valida disponibilidad, estado del responsable, conflictos y campos requeridos.
- 3 La solicitud se guarda con staffId y staffCode.
- 4 Se registra un evento en activityLog.
- 5 Se notifica al responsable y al cliente.
- 6 Los paneles se actualizan usando datos filtrados por rol.

6. Frontend y experiencia de usuario

El frontend debe mostrar una experiencia distinta por rol sin duplicar reglas criticas. La navegacion puede ser SPA simple con rutas virtuales o framework moderno. Lo importante es mantener la UI sincronizada con la sesion real del servidor.

Vistas esenciales

- Pagina publica: propuesta de valor, catalogo de servicios, llamada a accion y acceso a login.
- Login unificado: formulario unico con seleccion de rol o deteccion por usuario.
- Panel admin: KPIs, gestion de usuarios, solicitudes, calendario, pagos, notificaciones y reportes.
- Panel equipo: agenda propia, tareas asignadas, estados, perfil y mensajes.
- Panel cliente: historial, proximas solicitudes, pagos, perfil y preferencias.
- Formulario publico: creacion de solicitud sin cuenta, si el negocio lo permite.

Reglas de UI

- No mostrar botones que el rol no puede usar, pero tampoco confiar en eso como seguridad.
- Las tablas deben tener estados vacios, loading, error, filtros y acciones claras.
- Los formularios deben validar campos antes de enviar y mostrar errores del servidor sin esconderlos.
- Despues de login/logout, limpiar estado local y recargar datos desde /api/bootstrap.
- Evitar cache agresivo para JS durante desarrollo o despliegues frecuentes. Usar versionado de assets si hay CDN.

7. Seguridad minima y seguridad avanzada

Un sistema empresarial maneja datos personales, pagos, horarios, notas internas y actividad operativa. La seguridad no puede depender de ocultar botones o confiar en el navegador.

Seguridad minima obligatoria

- HTTPS obligatorio en produccion.
- Contraseñas hasheadas con PBKDF2, bcrypt o Argon2. Nunca texto plano.
- Cookies de sesion con HttpOnly, Secure y SameSite=Lax o Strict.
- Validacion de permisos en cada endpoint del servidor.
- Sanitizar datos enviados al frontend. No exponer passwordHash, tokens internos ni secretos.
- Bloquear acceso web a carpetas privadas como /storage, /.env o archivos ocultos.
- Registrar eventos importantes: login, fallo de login, creacion, edicion, cancelacion, pago y cambios de permisos.
- Backups automaticos de datos criticos.
- Variables de entorno para secretos, rutas, claves de pago y credenciales de emergencia.

Mejoras de seguridad recomendadas

Mejora	Por que importa	Prioridad
Rate limiting	Reduce ataques de fuerza bruta al login y formularios publicos.	Alta
2FA para admin	Protege acceso administrativo aun si una contrasena se filtra.	Alta
CSRF tokens	Protege acciones sensibles cuando se usan cookies.	Media
Politica de contrasenas	Exige longitud minima, rotacion de credenciales temporales y bloqueo por intentos.	Alta
Cifrado de datos sensibles	Protege notas, documentos, identificaciones o informacion financiera.	Media
Auditoria inmutable	Evita manipulacion de historial operativo o financiero.	Media
Base de datos gestionada	Mejora integridad, concurrencia, backups y escalabilidad.	Alta para produccion real

8. Despliegue en Hostinger

Hostinger puede servir aplicaciones Node.js si el plan lo permite. El despliegue debe configurarse para que el proceso Node arranque con el archivo correcto, tenga acceso a storage persistente y use variables de entorno.

Configuracion recomendada

- 1 Subir el repositorio o conectar GitHub desde hPanel.
- 2 Configurar la aplicacion Node.js con server.js como entry point.
- 3 Usar Node.js 18 o superior.
- 4 Definir script de inicio: npm start o node server.js.
- 5 Configurar dominio y HTTPS.
- 6 Crear una ruta persistente para datos, por ejemplo /storage/cd si Hostinger la permite.
- 7 Si la ruta absoluta no es escribible, usar fallback dentro de la carpeta Node, por ejemplo nodejs/storage/cd.
- 8 Bloquear acceso publico a /storage usando servidor Node y reglas .htaccess si aplica.
- 9 Reiniciar la app Node despues de cada deploy que cambie servidor, rutas o variables de entorno.

Variables de entorno sugeridas

```
PORT=8123
NODE_ENV=production
APP_URL=https://empresa.com
STORAGE_DIR=/home/usuario/domains/empresa.com/nodejs/storage/cd
ADMIN_PASSWORD=usar_un_secreto_largo
SESSION_SECRET=usar_un_token_largo
PAYMENT_PROVIDER_KEY=clave_del_proveedor
EMAIL_API_KEY=clave_de_email
SMS_API_KEY=clave_de_sms
```

Problemas comunes en Hostinger

Sintoma	Causa probable	Solucion
503	La app Node no arranco o se cayo al iniciar.	Revisar logs, entry point, version de Node y permisos de storage.

Sintoma	Causa probable	Solucion
Login falla tras deploy	JS viejo en cache, rol incorrecto, servidor no reiniciado o storage con datos antiguos.	Forzar no-cache, reiniciar Node, validar /api/auth/login con curl.
Datos desaparecen	Storage dentro de carpeta temporal o deploy sobrescribe archivos.	Usar ruta persistente fuera del build y backups.
Archivos privados visibles	Carpeta storage servida como estatica.	Bloquear /storage desde Node y reglas del servidor web.
API responde HTML	Fallback SPA captura rutas API.	Procesar /api/ antes del fallback estatico.

9. Pruebas, observabilidad y mantenimiento

Un sistema empresarial necesita pruebas practicas, no solo revision visual. Cada flujo principal debe verificarse con API y navegador.

Checklist de pruebas antes de deploy

- node --check server.js para validar sintaxis del servidor.
- node --check js/app.js y node --check js/auth.js para validar scripts principales.
- Login admin, equipo y cliente.
- Logout y segundo login con el mismo usuario.
- Creacion de usuario de equipo y verificacion de identificador operativo.
- Creacion de solicitud asignada y visibilidad correcta en panel del responsable.
- Edicion/cancelacion de solicitud con permisos correctos.
- Acceso bloqueado a rutas privadas sin sesion.
- Acceso bloqueado a /storage desde navegador.
- Prueba de /api/health en produccion.

Observabilidad minima

- Endpoint /api/health con estado del servidor y storage activo.
- Logs de errores de arranque.
- Registro de acciones en activityLog.
- Backups por fecha para colecciones criticas.
- Alertas futuras por email o dashboard cuando fallan pagos, login o escritura.

Migracion futura a base de datos

JSON puede funcionar para prototipos y negocios pequenos con bajo trafico. Cuando el sistema tenga concurrencia real, multiples sedes o datos financieros criticos, migrar a PostgreSQL, MySQL o una base gestionada. Mantener IDs, timestamps y colecciones limpias facilita esa transicion.

10. Mejoras futuras para optimizar empresas

El sistema puede evolucionar desde una herramienta operativa simple hacia una plataforma de optimizacion empresarial. Estas funciones agregan valor para empresas medianas sin rehacer la arquitectura base.

Automatizacion de tareas

Reglas para asignar responsables, enviar recordatorios, cambiar estados y escalar retrasos.

CRM ligero

Seguimiento de clientes, oportunidades, historial, etiquetas, segmentos y proxima accion.

Inventario

Control de stock, alertas de bajo inventario, compras y consumo asociado a servicios.

Facturacion

Recibos, facturas, impuestos, conciliacion y exportacion contable.

Analitica avanzada

Ingresos por periodo, productividad, tiempos promedio, retencion, cancelaciones y demanda.

Multi-sucursal

Separar sedes, horarios, personal, inventario, reportes y permisos por ubicacion.

Portal de proveedores

Pedidos, entregas, documentos, pagos pendientes y comunicacion con proveedores.

Asistente IA

Resumen de actividad, sugerencias de agenda, respuestas a clientes y deteccion de problemas.

Ruta de producto recomendada

- 1 MVP: login, roles, usuarios, solicitudes, agenda, notificaciones basicas y backups.
- 2 Operacion real: pagos, reportes, auditoria, permisos finos y calendario robusto.
- 3 Escalabilidad: base de datos, multi-sucursal, colas de trabajo y jobs programados.
- 4 Inteligencia: predicciones, automatizaciones, recomendaciones y analisis de rendimiento.
- 5 Plataforma: plantillas por industria, configuracion sin codigo y marketplace de integraciones.

11. Instrucciones para otra IA

Usa este bloque como especificacion para pedirle a otra IA que construya un sistema similar. La IA debe adaptar nombres y procesos al negocio especifico, pero conservar la arquitectura, seguridad y separacion de responsabilidades.

Construye una aplicacion web empresarial modular para una PYME.

Objetivo:

- Digitalizar procesos operativos como solicitudes, reservas, tareas, clientes, equipo, pagos, notificaciones, agenda y reportes.
- El sistema debe ser adaptable a diferentes industrias. No amarres el codigo a una sola categoria de negocio.

Arquitectura:

- Node.js como servidor principal.
- Frontend web con HTML/CSS/JS o framework moderno.
- API interna bajo /api.
- Sesiones con cookies HttpOnly.
- Roles: admin, staff/equipo, customer/cliente.
- Storage persistente en Hostinger. Usar JSON para MVP, pero disenar para migrar a SQL.

Modulos:

- Login/logout y proteccion por rol.
- Panel admin con usuarios, solicitudes, calendario, pagos, notificaciones, reportes y settings.
- Panel de equipo con tareas propias, agenda y perfil.
- Panel de cliente con historial, solicitudes, pagos y perfil.
- Formulario publico opcional para solicitudes sin cuenta.
- Logs de actividad y backups.

Seguridad:

- Nunca guardar contraseñas en texto plano.
- No enviar passwordHash al frontend.
- Validar permisos en servidor, no solo en UI.
- Bloquear acceso a storage, .env y archivos ocultos.
- Usar HTTPS, cookies Secure, HttpOnly y SameSite.
- Agregar rate limiting, 2FA admin y CSRF tokens en version avanzada.

Hostinger:

- Entry point: server.js.
- Script: npm start o node server.js.
- Node >= 18.
- Usar ruta persistente para storage.
- Si /storage/cd no es escribible, usar fallback dentro de nodejs/storage/cd.
- Reiniciar la app Node despues de cada deploy.
- Verificar /api/health y /api/auth/login despues de subir cambios.

Calidad:

- Crear checklist de pruebas para login, logout, creacion de usuarios, asignacion de tareas, visibilidad por rol, pagos, notificaciones y bloqueo de rutas privadas.
- Mantener documentacion de estructura, endpoints, variables de entorno y tareas de deploy.

Criterios de aceptacion

- Un admin puede entrar, salir y volver a entrar sin error de rol.
- Un usuario de equipo ve solo sus tareas o solicitudes asignadas.
- El admin ve todas las solicitudes y puede reasignarlas.
- El cliente ve solo sus propios datos.
- Las contraseñas nunca aparecen en respuestas API.
- El sistema arranca correctamente en Hostinger y responde /api/health.
- Los datos persisten despues de reiniciar la app.
- Existe una estrategia clara de backups y migracion futura a base de datos.

Resumen ejecutivo

La forma mas practica de construir este tipo de sistema es comenzar con una arquitectura simple y disciplinada: Node.js, API propia, frontend por roles, storage persistente, sesiones seguras, logs y backups. La clave no es tener muchos modulos desde el primer dia, sino que cada modulo respete permisos, datos limpios y flujos verificables.

Cuando el producto gane usuarios, la evolucion natural es migrar la persistencia a SQL, agregar integraciones externas, automatizaciones y analitica. Si la base se construye con entidades genericas y seguridad en servidor, el mismo codigo puede adaptarse a multiples industrias y empresas sin rehacer la plataforma.